Departamento de Computação — DCOMP/UFS
COMP0511 — Programação B

Fundamentos de React Native e primeiras telas

Roteiro prático

Prof. Dr. André Yoshiaki Kashiwabara — andreyoshiaki@dcomp.ufs.br

Nome(s): _____ Data: _____

Este roteiro acompanha os slides. Tudo aqui roda no navegador e no celular: não há nada para instalar no computador. Os códigos que você vai colar no Snack estão no Apêndice (p. 8).

Ao final da aula você será capaz de

- explicar por que o `<Text>` do React Native vira um componente nativo do Android ou do iOS;
- escrever arrow functions, desestruturar objetos e vetores e transformar listas com `map`;
- ler e escrever JSX, criar um componente e passar dados por props;
- montar uma tela com `View`, `Text`, `Image`, `TextInput` e `Pressable`, estilizada com `StyleSheet` e posicionada com Flexbox;
- usar `useState` para que a tela reaja ao toque e ao que o usuário digita.

Antes de começar

1. No **celular**: instale o aplicativo **Expo Go** (Play Store ou App Store).
2. No **computador**: abra `https://snack.expo.dev` em uma aba nova.
3. Na barra da direita do Snack, escolha **My Device**. Vai aparecer um QR code.
4. Abra o Expo Go e leia o QR code. No iPhone, use a câmera do sistema.
5. Guarde a URL que o Snack gerou — é nela que o seu trabalho de hoje vai crescer.

> **Se a rede bloquear a conexão:** use o botão **Web** do próprio Snack. A tela roda no navegador. Você perde o celular de verdade, mas nenhum conceito da aula.

1 O JavaScript que a tela exige

Arrow functions

Em JavaScript, funções são valores: dá para guardá-las em variáveis e passá-las como argumento — que é exatamente o que um botão precisa receber (Flanagan, 2020, §8.1.3).

```
function dobro(x) {            // forma tradicional
  return x * 2;
}

const dobro = (x) => x * 2;    // a mesma coisa; sem chave, o return é implícito

const saudacao = (nome) => {   // com chave, volta o bloco e o return explícito
  const texto = 'Olá, ' + nome;
  return texto;
};
```

Desestruturação

O lado esquerdo da atribuição imita o formato do dado: cada variável recebe o campo de mesmo nome (objeto) ou o valor na mesma posição (vetor) (Flanagan, 2020, §3.10).

```
const aluno = { nome: 'Ada', curso: 'Programação B', nota: 9.5 };

const { nome, nota } = aluno;   // tira campos de dentro do objeto
const [a, b] = [10, 20];        // em vetores, por posição
```

A forma de vetor é a que o `useState` usa: `const [valor, setValor] = useState(0)`.

`map`

`map` aplica uma função a cada item e devolve um vetor novo, do mesmo tamanho (Flanagan, 2020, §7.8.1).

```
const habilidades = ['JavaScript', 'React Native', 'Flexbox'];
const tamanhos = habilidades.map((h) => h.length);   // [10, 12, 7]
```

Antes de rodar, escreva a sua resposta

Quanto vale `resultado`?

```
const notas = [7, 5, 9, 4];
const resultado = notas.map((n) => {
  n >= 6;
});
```

Minha resposta:

Depois de responder, confira na p. 12.

JSX, componente e props

Um componente é uma função com nome **maiúsculo** que devolve JSX; as props chegam como um único objeto (Banks; Porcello, 2020, cap. 5).

```
function Saudacao({ nome, horario }) {          // props chegam como um objeto
  return <Text>Bom {horario}, {nome}!</Text>;
}

<Saudacao nome="Ada" horario="dia" />           // quem chama decide os valores
```

Quatro regras de JSX que sempre pegam:

1. chaves `{ }` entram em modo JavaScript;
2. só pode haver **um** elemento raiz;
3. toda tag fecha, inclusive as vazias: `<Image source={foto} />`;
4. estilo é objeto, não string: `style={{ fontSize: 20 }}`.

2 A primeira tela

Apague o conteúdo do `App.js` do Snack e digite (ou cole) o programa abaixo:

```
import { View, Text, StyleSheet } from 'react-native';

export default function App() {
  return (
    <View style={estilos.tela}>
      <Text style={estilos.titulo}>Olá, COMP0511!</Text>
    </View>
  );
}

const estilos = StyleSheet.create({
  tela: { flex: 1, justifyContent: 'center', alignItems: 'center' },
  titulo: { fontSize: 28, fontWeight: 'bold' },
});
```

Tarefa 1 — 5 minutos

- ☐ Ponha o código acima para rodar no seu aparelho.
- ☐ Troque o texto pelo seu próprio nome.
- ☐ Mude `fontSize` para `48` e veja a tela recarregar sozinha.
- ☐ **Apague a linha `flex: 1`.** Para onde o texto foi? Por quê?

Quando não funciona

Sintoma	Causa provável
O QR code não conecta	Celular e computador em redes diferentes — use **Web**
Tela branca, sem erro	Faltou `export default` antes de `function App`
`Text strings must be rendered within a <Text>`	Texto solto dentro de uma `View`
A imagem não aparece	`Image` com `uri` exige `width` e `height` no estilo

3 Componentes, estilo e Flexbox

Componente	Equivale a	Para que serve
`View`	`<div>`	caixa: agrupa, posiciona, recebe cor e borda
`Text`	`<p>`	todo texto visível — nada de texto solto
`Image`	`<img>`	imagem local ou vinda de uma URL
`TextInput`	`<input>`	entrada de texto do usuário
`Pressable`	`<button>`	qualquer área que reage ao toque

Rode o programa **A.1 (catálogo de componentes)** do apêndice e experimente:

- ☐ troque `flexDirection: 'row'` do estilo `linha` para `'column'`;
- ☐ em `tela`, troque `alignItems: 'center'` por `'flex-start'` e depois `'flex-end'`;
- ☐ acrescente `justifyContent: 'space-between'` em `linha`.

As três propriedades que resolvem quase todo layout (Eisenman, 2017, cap. 5):

- **`flexDirection`** escolhe o eixo principal (`'column'` é o padrão no React Native);
- **`justifyContent`** distribui no eixo principal;
- **`alignItems`** alinha no eixo cruzado.

> **Diferenças em relação ao CSS da web:** o padrão é `column` (e não `row`), `flex` é um número só, não há unidades (`px`, `%`) e as propriedades usam *camelCase* (`backgroundColor`, não `background-color`).

4 Telas que reagem: `useState`

Antes de rodar, escreva a sua resposta

Você toca **cinco** vezes no botão. O que a tela mostra?

```
export default function App() {
  let contagem = 0;
  return (
    <View>
      <Text>{contagem}</Text>
      <Pressable onPress={() => { contagem = contagem + 1; }}>
        <Text>somar 1</Text>
      </Pressable>
    </View>
  );
}
```

Minha resposta:

Depois de responder, confira na p. 12.

O `useState` **lembra** um valor entre as renderizações e **avisa** o React que a tela precisa ser redesenhada (Banks; Porcello, 2020, cap. 6):

```
const [contagem, setContagem] = useState(0);
//     |           |                   +-- valor inicial (só na 1ª vez)
//     |           +---------- troca o valor E pede um novo desenho
//     +----------------------- valor atual, válido nesta renderização
```

Rode os dois exemplos do apêndice e leia o código linha a linha:

- ☐ **A.2 — contador**;
- ☐ **A.3 — campo controlado**.

No campo controlado, o `TextInput` não guarda nada. A prop `value={nome}` faz o campo mostrar o estado; já `onChangeText={setNome}` avisa quando o usuário pede uma mudança.

Três armadilhas

```
setContagem(contagem + 1);
console.log(contagem);          // 1. ainda o valor ANTIGO

setContagem((n) => n + 1);      // 2. quando o novo valor depende do antigo

lista.push('novo');             // 3. ERRADO: React não percebe a mudança
setLista([...lista, 'novo']);   //    CERTO: um vetor novo
```

5 Tarefa 2 — o cartão de perfil

Trabalhe em **duplas**, no mesmo Snack, partindo do programa **A.4 — esqueleto do cartão** do apêndice. **Cada checkpoint tem que rodar no celular antes de você começar o próximo.** Marque a caixa quando o item funcionar.

Checkpoint 1 — o cartão estático

- ☐ cabeçalho em linha: foto redonda de 72×72 à esquerda; ao lado, em coluna, o nome e o usuário;
- ☐ a bio abaixo;
- ☐ as habilidades como etiquetas, uma por item do vetor, geradas com `map`.

Dicas: `flexDirection: 'row'` + `alignItems: 'center'` no cabeçalho; `borderRadius: 36` para uma foto de 72 de lado; cada elemento produzido por `map` precisa de uma prop `key` única.

Checkpoint 2 — o contador de curtidas

- ☐ um `Pressable` no rodapé do cartão mostrando `0 curtidas`;
- ☐ cada toque soma 1;
- ☐ o texto concorda em número: `1 curtida`, `2 curtidas`.

Use a forma funcional: `setCurtidas((n) => n + 1)`.

Checkpoint 3 — a bio vira campo controlado

- ☐ crie um estado `bio` iniciado com `PERFIL.bio`;
- ☐ o `<Text>` da bio passa a mostrar o estado, não mais a constante;
- ☐ acrescente um `TextInput` com `value={bio}` e `onChangeText={setBio}`;
- ☐ mostre um contador `N/120` abaixo do campo.

Ao digitar, o texto do cartão muda junto — é o mesmo estado em dois lugares.

Checkpoint 4 — Seguir / Seguindo

- ☐ estado booleano `seguindo`, começando em `false`;
- ☐ o botão mostra `Seguir` ou `Seguindo` e troca de cor: `style={[estilos.botaoSeguir, seguindo && estilos.botaoSeguindo]}`;
- ☐ o total de seguidores sobe 1 quando você está seguindo.

Atenção ao último item: o total de seguidores **não** é um estado novo. Ele se calcula a partir de `seguindo`:

```
const seguidores = seguindo ? PERFIL.seguidores + 1 : PERFIL.seguidores;
```

Estado duplicado é fonte de inconsistência: dois lugares para atualizar, um deles esquecido.

6 Desafio para casa

Transforme o cartão de perfil em uma **lista de contatos**:

1. extraia um componente `Cartao` que receba `nome`, `usuario`, `foto` e `habilidades` por props;
2. monte um vetor com **quatro** pessoas e renderize os quatro cartões com `map`;
3. cada cartão mantém o **próprio** contador de curtidas — curtir um não pode afetar os outros;
4. acrescente um `TextInput` no topo que filtra os cartões pelo nome.

Pergunta que decide o item 3: onde deve morar o estado das curtidas — no `App` ou dentro do `Cartao`?

Dica para o item 4:

```
lista.filter((p) => p.nome.toLowerCase().includes(busca.toLowerCase()))
```

Referências

- Banks, A.; Porcello, E. *Learning React*. 2. ed. O'Reilly, 2020 — cap. 5 (JSX) e cap. 6 (estado).
- Eisenman, B. *Learning React Native*. 2. ed. O'Reilly, 2017 — cap. 1 (panorama) e cap. 5 (estilo).
- Flanagan, D. *JavaScript: The Definitive Guide*. 7. ed. O'Reilly, 2020 — §3.10, §7.8.1, §8.1.3.
- React Native Docs — `https://reactnative.dev/docs/getting-started`
- React Docs — `https://react.dev/learn`
- Expo Docs — `https://docs.expo.dev/`

Apêndice — programas para colar no Snack

Ao copiar do PDF, a indentação pode se perder; isso não muda o funcionamento do JavaScript.

A.1 — Catálogo de componentes

```
// Catálogo mínimo: View, Text, Image, TextInput, Pressable.
// Nada aqui guarda estado ainda --- o campo digita e o botão não reage.
import { View, Text, Image, TextInput, Pressable, StyleSheet } from 'react-native';

export default function App() {
  return (
    <View style={estilos.tela}>
      <Text style={estilos.titulo}>Catálogo de componentes</Text>

      {/* Image remota EXIGE width e height no estilo */}
      <Image
        style={estilos.foto}
        source={{ uri: 'https://picsum.photos/id/1005/200/200' }}
      />

      <TextInput style={estilos.campo} placeholder="Digite seu nome" />

      <Pressable
        style={estilos.botao}
        onPress={() => console.log('O botão foi tocado')}
      >
        <Text style={estilos.textoBotao}>Tocar aqui</Text>
      </Pressable>

      {/* Uma linha: flexDirection 'row' troca o eixo principal */}
      <View style={estilos.linha}>
        <View style={[estilos.quadrado, { backgroundColor: '#e63946' }]} />
        <View style={[estilos.quadrado, { backgroundColor: '#457b9d' }]} />
        <View style={[estilos.quadrado, { backgroundColor: '#2a9d8f' }]} />
      </View>
    </View>
  );
}

const estilos = StyleSheet.create({
  tela: { flex: 1, alignItems: 'center', paddingTop: 64, backgroundColor: '#fff' },
  titulo: { fontSize: 22, fontWeight: 'bold', marginBottom: 16 },
  foto: { width: 120, height: 120, borderRadius: 60 },
  campo: {
    width: 240, height: 44, marginTop: 20, paddingHorizontal: 12,
    borderWidth: 1, borderColor: '#c9d2dd', borderRadius: 8,
  },
  botao: {
    marginTop: 16, paddingVertical: 12, paddingHorizontal: 24,
    backgroundColor: '#12263f', borderRadius: 8,
  },
  textoBotao: { color: '#fff', fontWeight: 'bold' },
  linha: { flexDirection: 'row', gap: 12, marginTop: 24 },
  quadrado: { width: 56, height: 56, borderRadius: 8 },
});
```

A.2 — Contador

```
// Contador: o menor programa que mostra o estado mudando a tela.
import { useState } from 'react';
import { View, Text, Pressable, StyleSheet } from 'react-native';

export default function App() {
  // useState devolve um par: o valor atual e a função que o troca.
  const [contagem, setContagem] = useState(0);

  return (
    <View style={estilos.tela}>
      <Text style={estilos.numero}>{contagem}</Text>

      <View style={estilos.linha}>
        <Pressable style={estilos.botao} onPress={() => setContagem(contagem - 1)}>
          <Text style={estilos.textoBotao}>-1</Text>
        </Pressable>

        {/* Forma funcional: use quando o novo valor depende do anterior */}
        <Pressable style={estilos.botao} onPress={() => setContagem((n) => n + 1)}>
          <Text style={estilos.textoBotao}>+1</Text>
        </Pressable>
      </View>

      <Pressable onPress={() => setContagem(0)}>
        <Text style={estilos.zerar}>zerar</Text>
      </Pressable>
    </View>
  );
}

const estilos = StyleSheet.create({
  tela: { flex: 1, justifyContent: 'center', alignItems: 'center', backgroundColor: '#fff' },
  numero: { fontSize: 72, fontWeight: 'bold', color: '#12263f' },
  linha: { flexDirection: 'row', gap: 16, marginTop: 24 },
  botao: {
    width: 72, height: 56, borderRadius: 10, backgroundColor: '#12263f',
    justifyContent: 'center', alignItems: 'center',
  },
  textoBotao: { color: '#fff', fontSize: 20, fontWeight: 'bold' },
  zerar: { marginTop: 20, color: '#5b6b7f', textDecorationLine: 'underline' },
});
```

A.3 — Campo controlado

```
// Campo controlado: o estado manda no que aparece dentro do TextInput.
import { useState } from 'react';
import { View, Text, TextInput, StyleSheet } from 'react-native';

export default function App() {
  const [nome, setNome] = useState('');

  return (
    <View style={estilos.tela}>
      <Text style={estilos.rotulo}>Seu nome</Text>

      <TextInput
        style={estilos.campo}
```

```
        placeholder="digite aqui"
        value={nome}              // o que aparece vem do estado
        onChangeText={setNome}    // cada tecla devolve o texto novo ao estado
      />

      <Text style={estilos.saida}>
        {nome.length === 0 ? 'Ainda não digitou nada.' : `Olá, ${nome}!`}
      </Text>
      <Text style={estilos.contador}>{nome.length}/30 caracteres</Text>
    </View>
  );
}

const estilos = StyleSheet.create({
  tela: { flex: 1, justifyContent: 'center', paddingHorizontal: 32, backgroundColor: '#fff' },
  rotulo: { fontSize: 14, color: '#5b6b7f', marginBottom: 6 },
  campo: {
    height: 48, paddingHorizontal: 12, fontSize: 16,
    borderWidth: 1, borderColor: '#c9d2dd', borderRadius: 8,
  },
  saida: { fontSize: 22, fontWeight: 'bold', marginTop: 24, color: '#12263f' },
  contador: { fontSize: 12, color: '#8896a6', marginTop: 4 },
});
```

A.4 — Esqueleto do cartão de perfil

```jsx
// ============================================================
// Cartão de perfil --- ponto de partida
// Cole em snack.expo.dev, leia o QR com o Expo Go e trabalhe aqui.
// Os TODO marcam o que é seu. Não apague os dados abaixo.
// ============================================================
import { useState } from 'react';
import { View, Text, Image, TextInput, Pressable, StyleSheet } from 'react-native';

const PERFIL = {
  nome: 'Ada Lovelace',
  usuario: '@ada',
  foto: 'https://picsum.photos/id/1027/200/200',
  bio: 'Estudante de Programação B. Gosto de algoritmos e de café.',
  habilidades: ['JavaScript', 'React Native', 'Flexbox', 'C'],
  seguidores: 128,
};

export default function App() {
  // CHECKPOINT 2 --- TODO: estado das curtidas, começando em 0
  // CHECKPOINT 3 --- TODO: estado da bio, começando em PERFIL.bio
  // CHECKPOINT 4 --- TODO: estado booleano "seguindo", começando em false

  return (
    <View style={estilos.tela}>
      <View style={estilos.cartao}>

        {/* CHECKPOINT 1 --- TODO: cabeçalho em linha:
            a foto (redonda, 72x72) à esquerda; ao lado, em coluna,
            o nome e o usuário. */}

        {/* CHECKPOINT 3 --- TODO: trocar o texto fixo pelo estado da bio,
            e acrescentar um TextInput controlado que a edita. */}
        <Text style={estilos.bio}>{PERFIL.bio}</Text>

        {/* CHECKPOINT 1 --- TODO: as habilidades, uma etiqueta por item,
            geradas com map. Cada elemento da lista precisa de key. */}

        {/* CHECKPOINT 2 --- TODO: botão de curtir mostrando o total.
            CHECKPOINT 4 --- TODO: botão Seguir / Seguindo ao lado. */}

      </View>
    </View>
  );
}

const estilos = StyleSheet.create({
  tela: { flex: 1, justifyContent: 'center', paddingHorizontal: 20, backgroundColor: '#eef1f6' },
  cartao: {
    backgroundColor: '#fff', borderRadius: 16, padding: 20,
    // sombra leve; não precisa entender agora
    shadowColor: '#000', shadowOpacity: 0.12, shadowRadius: 12,
    shadowOffset: { width: 0, height: 4 }, elevation: 4,
  },
  bio: { fontSize: 14, color: '#43536b', lineHeight: 20, marginTop: 14 },
  // TODO: crie aqui os estilos que os seus checkpoints pedirem.
});
```

Respostas das previsões

Leia só depois de escrever a sua resposta.

`map` com chave

`[undefined, undefined, undefined, undefined]`. A chave depois da seta abre um bloco de comandos, e bloco de comandos exige `return`. Corrija de uma das duas formas:

```
notas.map((n) => { return n >= 6; });
notas.map((n) => n >= 6);
```

`let contagem`

Zero, sempre zero. A variável muda na memória, mas ninguém pede para redesenhar a tela; e se algo redesenhar, a função `App` roda de novo e `contagem` volta a `0`. Falta um mecanismo que **lembre** o valor entre as renderizações e **avise** que a tela mudou. Esse mecanismo é o `useState`.

Por que o texto saiu do centro sem `flex: 1`?

`justifyContent` distribui o conteúdo *no espaço que a caixa tem*. Sem `flex: 1` a `View` encolhe até caber no texto — e não sobra espaço para distribuir. O texto vai para o topo da tela.